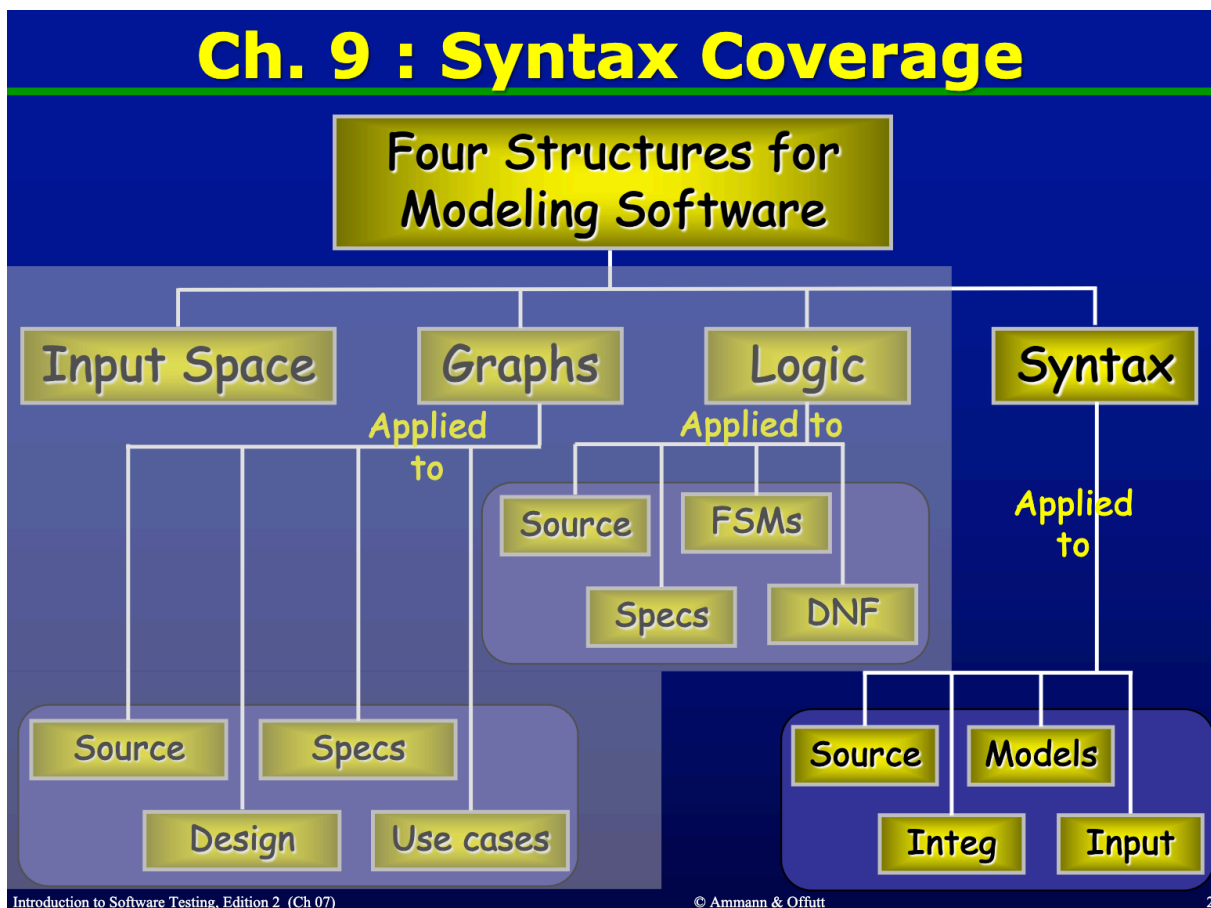




Chapter 9: Syntax Testing



🧠 Grammar-Based Testing (Ch 9.1 & 9.5)

◆ Core Idea

Use **syntax (grammar)** to generate test cases

→ Software inputs often follow **strict rules (grammar, e.g., BNF)**

2 main goals:

- Cover valid syntax
- Break syntax (invalid tests)

◆ Input Space Concept

Input Space = set of all allowable inputs

Ways to describe it:

- User manuals
- APIs / method signatures
- Input formats (XML, SQL, etc.)
- Grammars (most powerful)

👉 Testers often **create grammars themselves** → **find bugs early**

◆ Input Domain Layers (VERY IMPORTANT)

From slide diagram (p.7–9):

- **Goal domain** → what SHOULD be accepted
- **Specified domain** → what spec says
- **Implemented domain** → what code actually accepts

🔥 **Bugs live in mismatches between these**

◆ Invalid Input Testing

- Software must **reject or handle invalid inputs**
 - Many systems assume input is valid → ❌ dangerous
 - Major source of:
 - crashes
 - data corruption
 - security vulnerabilities
-

◆ Grammar Types

1. Regular Expressions

- Simple patterns
- Example: `(deposit account amount | debit account amount)*`

2. BNF (Backus-Naur Form)

- More expressive
- Uses rules like:

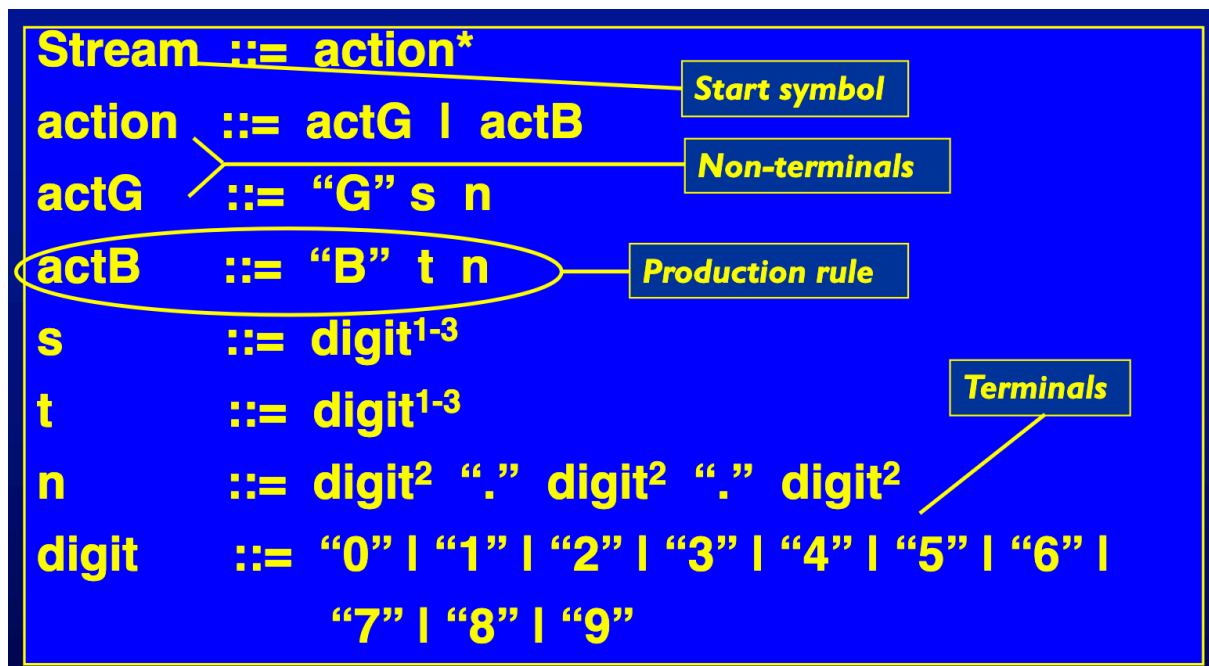
```
bank ::= action*  
action ::= dep | deb  
dep ::= "deposit" account amount
```

1. XML + Schema

- Real-world usage (web services)
- Schema = grammar + constraints

◆ Grammar Structure

- **Terminal** → actual symbols (e.g., "deposit", digits)
- **Non-terminal** → variables (e.g., account, amount)
- **Production rule** → how to expand
- **Start symbol** → entry point



Here's a quick example using a simple grammar for bank transactions:

```

<transaction> ::= <action> <account> <amount>
<action>      ::= "deposit" | "withdraw"
<account>     ::= "checking" | "savings"
<amount>      ::= <digit> <amount> | <digit>
<digit>       ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

```

Breaking down each piece:

Terminals — the literal symbols that actually appear in the final string. They can't be expanded further. Here: `"deposit"`, `"withdraw"`, `"checking"`, `"savings"`, `"0"` — `"9"`.

Non-terminals — variables/placeholders that get replaced via rules. Conventionally written in angle brackets or capitalized. Here: `<transaction>`, `<action>`, `<account>`, `<amount>`, `<digit>`.

Production rules — the `::=` lines telling you how to expand a non-terminal. For example, `<action> ::= "deposit" | "withdraw"` means "an action can be replaced by either the terminal `deposit` or the terminal `withdraw`." The rule `<amount> ::= <digit> <amount> | <digit>` is recursive — it lets amounts be one or more digits.

Start symbol — the entry point you begin derivations from. Here it's `<transaction>`.

A sample derivation starting from the start symbol:

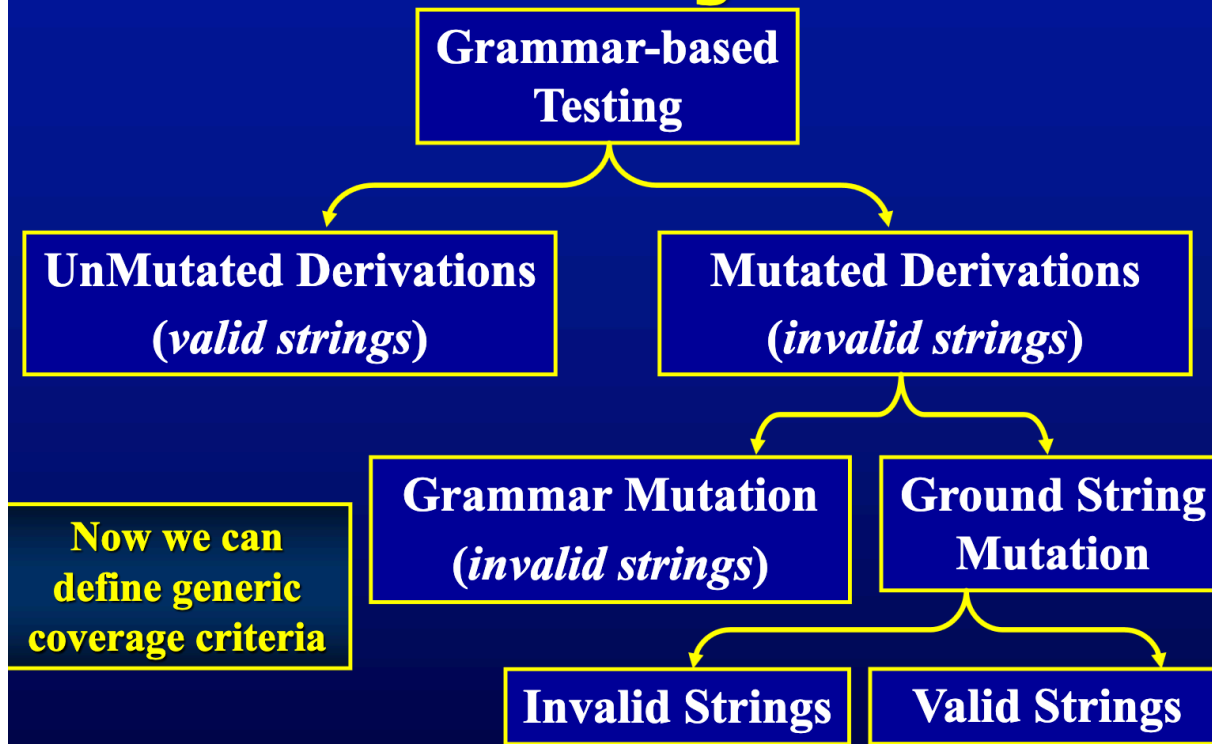
```

<transaction>
→ <action> <account> <amount>
→ "deposit" <account> <amount>
→ "deposit" "savings" <amount>
→ "deposit" "savings" <digit> <amount>
→ "deposit" "savings" "5" <digit>
→ "deposit" "savings" "50"

```

Final string (only terminals): `deposit savings 50`.

Mutation as Grammar-Based Testing



◆ Generating Tests

Valid tests:

- Derive strings using grammar rules

Invalid tests:

- Mutate grammar or break rules

Example derivation:

```
P ⇒ (P lop P)
⇒ (!a & (b | c))
```

◆ Coverage Criteria (VERY TESTABLE)

1. Terminal Symbol Coverage (TSC)

- Use every terminal at least once

2. Production Coverage (PDC)

- Use every rule at least once

👉 PDC > TSC (stronger)

3. Derivation Coverage (DC)

- All possible strings ✗ impractical

- The most common and straightforward criteria use every terminal and every production at least once (9.1.1)

Terminal Symbol Coverage (TSC) :TR contains each terminal symbol t in the grammar G .

Production Coverage (PDC) :TR contains each production p in the grammar G .

- PDC subsumes TSC
- Grammars and graphs are interchangeable
 - PDC is equivalent to EC, TSC is equivalent to NC
- Other graph-based coverage criteria could be defined on grammar
 - But have not

- A related criterion is the impractical one of deriving all possible strings

Derivation Coverage (DC) :TR contains every possible string that can be derived from the grammar G.

- The number of **TSC tests** is bound by the number of **terminal symbols**
 - 13 in the stream grammar
- The number of **PDC tests** is bound by the number of **productions**
 - 18 in the stream grammar
- The number of **DC tests** depends on the **details** of the grammar
 - 2,000,000,000 in the stream grammar !
- All TSC, PDC and DC tests are **in the grammar** ... how about tests that are **NOT in the grammar** ?

◆ Mutation Testing (KEY CONCEPT)

Definitions

- **Ground string** → valid string from grammar
- **Mutation operator** → rule to change string
- **Mutant** → modified version

Types

- Valid mutants (still in grammar)
- Invalid mutants (break grammar)

Killing Mutants

A test **kills a mutant** if:

Given a mutant m in M for a derivation D and a test t , t is said to kill m if and only if the output of t on D is different from the output of t on m

```
output(original) ≠ output(mutant)
```

Mutants and Ground Strings

- The key to mutation testing is the **design** of the mutation operators
 - Well designed **operators** lead to powerful testing
- Sometimes **mutant strings** are based on ground strings
- Sometimes they are derived directly **from the grammar**
 - **Ground** strings are used for **valid** tests
 - **Invalid** tests do not need ground strings

Valid Mutants

Ground Strings

G 26 08.01.90

B 22 06.27.94

Mutants

B 26 08.01.90

B 45 06.27.94

Invalid Mutants

7 26 08.01.90

B 22 06.27.1

Coverage

- **Mutation Coverage (MC)**
 - kill all mutants
- **Mutation Operator Coverage (MOC)**
 - use each mutation operator once
- **Mutation Production Coverage (MPC)**
 - mutate each production rule

- Coverage is defined in terms of killing mutants

Mutation Coverage (MC) : For each $m \in M$, TR contains exactly one requirement, to kill m .

- Coverage in mutation equates to number of mutants killed
- The amount of mutants killed is called the **mutation score**

- When creating invalid strings, we just apply the operators
- This results in two simple criteria
- It makes sense to either use every operator once or every production once

Mutation Operator Coverage (MOC) : For each mutation operator, TR contains exactly one requirement, to create a mutated string m that is derived using the mutation operator.

Mutation Production Coverage (MPC) : For each mutation operator, TR contains several requirements, to create one mutated string m that includes every production that can be mutated by that operator.

◆ Big Picture

Grammar-based testing =

Type	What it tests
Valid strings	correctness
Invalid strings	robustness
Mutants	fault detection

◆ Key Insights (Exam Gold)

- Grammar = **model of input space**
 - Testing = **generate + break grammar**
 - Bugs = **gap between spec & implementation**
 - Mutation = **most powerful but expensive**
 - PDC $\approx$ Edge Coverage, TSC $\approx$ Node Coverage
-

◆ One-line Memory Hook

👉 *"Grammar defines inputs $\rightarrow$ tests are strings $\rightarrow$ mutation breaks them to expose bugs."*
